

Universidad de Antioquia

Facultad de Ingeniería — Departamento de Ingeniería de Sistemas

Solver de Programación Lineal

Resolución paso a paso, métodos Simplex, Gran M, Gráfico y análisis de sensibilidad

Curso · Optimización 2026-1

Trabajo · Programación Lineal — Entrega final

Fecha · 30 de mayo de 2026

Equipo

Gabriel Jaime Cardona Montoya

Juan Sebastián Tabares

Víctor Rodas

Repositorio público: github.com/gjcardonam/optimizacion-pl-udea

1. Descripción del programa

El programa es un **solver interactivo de Programación Lineal** desarrollado en **Python 3.13** con interfaz web en **Streamlit**. Recibe el problema en su **forma básica** (antes de la aumentada), lo convierte automáticamente a forma estándar y muestra la solución **paso a paso, tablero por tablero**, replicando el formato de las diapositivas del curso.

El sistema escoge automáticamente el método apropiado:

- **Simplex tabular básico** cuando todas las restricciones son del tipo $\leq$ (la base inicial trivial con holguras)

es factible).

- **Método de la Gran M** cuando existen restricciones ≥ 0 = (se requieren variables artificiales penalizadas).

Adicionalmente expone el **Simplex Revisado** en forma matricial ($B, B^{-1}, c_B \cdot B^{-1}$), el **método gráfico** para problemas de 2 variables y un **análisis de sensibilidad** post-óptimo con precios sombra, costos reducidos y rangos de variación.

2. Problema modelo

Para validar el comportamiento del programa se usa como problema de referencia el clásico de **Wyndor Glass** (Hillier & Lieberman):

*Wyndor Glass debe decidir cuántos lotes producir de dos productos nuevos (puertas y ventanas) para **maximizar la utilidad**, sujeto a la disponibilidad de tiempo en tres plantas de producción.*

Formulación matemática:

$$\text{MAX } Z = 3x_1 + 5x_2$$

sujeto a:

$$x_1 \leq 4 \quad (\text{Planta 1})$$

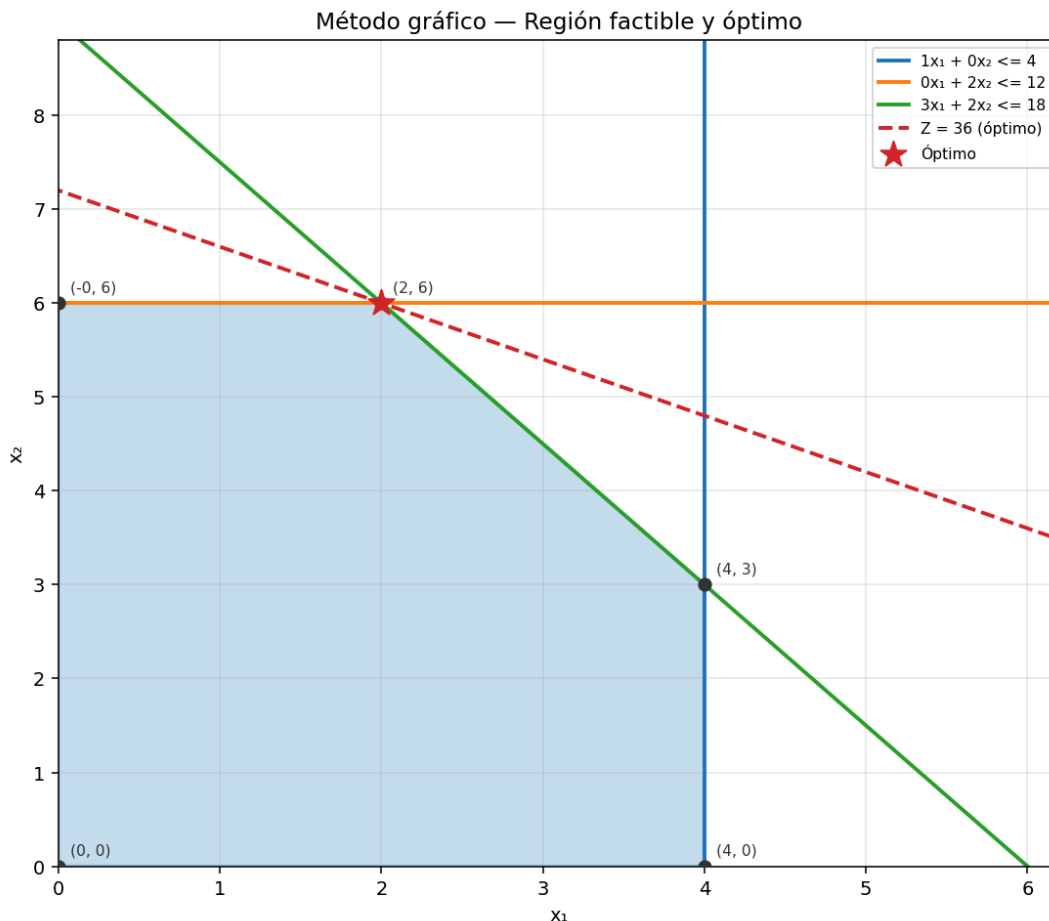
$$2x_2 \leq 12 \quad (\text{Planta 2})$$

$$3x_1 + 2x_2 \leq 18 \quad (\text{Planta 3})$$

$$x_1, x_2 \geq 0$$

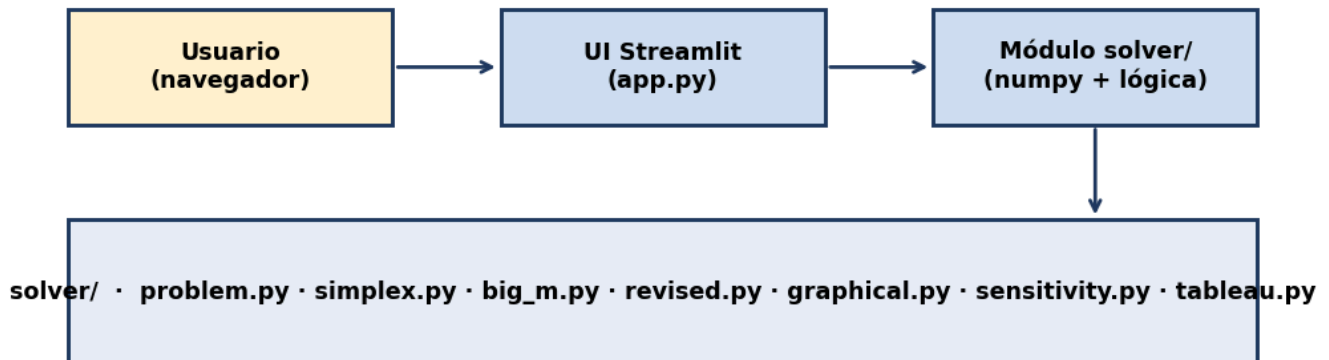
Solución óptima (verificada con todos los métodos): $x_1 = 2, x_2 = 6, Z^* = 36$.

El método gráfico generado automáticamente por el programa es:



3. Arquitectura

El proyecto separa la lógica del solver de la interfaz: el módulo `solver/` es independiente y reutilizable.



```
optimizacion-pl/
├── app.py                UI Streamlit
├── solver/
│   ├── problem.py       LPPProblem · conversión forma básica → estándar
│   ├── tableau.py       Representación de un tablero Simplex
│   ├── simplex.py       Simplex tabular básico
│   ├── big_m.py         Método de la Gran M
│   ├── revised.py       Simplex Revisado (forma matricial)
│   ├── graphical.py     Método gráfico (matplotlib)
│   └── sensitivity.py   Análisis de sensibilidad
├── tests/               Suite de validación
└── DOCUMENTO.md / .pdf Este entregable
```

4. Operaciones que realiza el programa

4.1 Lectura y normalización

El usuario ingresa el problema en forma básica (vector $\mathbf{c}$ y restricciones $(a_{ij}, \geq, b_i)$). El programa:

1. Si es minimización, multiplica $\mathbf{c}$ por -1 (resuelve internamente como max).
2. Si $b_i < 0$, multiplica toda la fila por -1 e invierte el sentido.
3. Agrega variables auxiliares: s_i (holgura) para $\leq$, e_i (exceso) + a_i (artificial) para $\geq$, a_i (artificial) para $=$.
4. Construye $\mathbf{A}$, $\mathbf{b}$, $\mathbf{c}$ extendido e identifica la base inicial.

4.2 Simplex tabular

Algoritmo Simplex clásico con regla de Dantzig para la entrada y razón mínima para la salida. Frente a empates usa **regla de Bland** (menor índice básico) para evitar ciclos. Detecta no acotado (columna sin coeficientes positivos) y óptimos múltiples (coef. reducido = 0 en variable no básica).

4.3 Gran M

Penaliza artificiales con $-M = -10^6$ en la función objetivo. Antes de iterar, **limpia la fila z** restando $M \times \text{fila}_i$ por cada artificial básica. Tras converger, si alguna artificial sigue con valor $> 0 \rightarrow$ **INFECTIBLE**.

4.4 Método gráfico

Para $n=2$: intersecciona cada par de rectas (incluidos ejes), filtra los vértices factibles, evalúa Z en cada uno y

selecciona el óptimo. Grafica con `matplotlib` la región factible, las rectas, los vértices y la curva de nivel $Z = Z^*$.

4.5 Simplex Revisado

Misma lógica pero expresada en forma matricial. Por iteración calcula y reporta:

Magnitud	Significado
B	Matriz base (columnas de A asociadas a la base actual)
B^{-1}	Inversa de la base
c_B	Coefficientes objetivo de las variables básicas
$x_B = B^{-1} \cdot b$	Valores actuales de las variables básicas
$y = c_B \cdot B^{-1}$	Vector dual (precios sombra)
$r_j = c_j - y \cdot A_j$	Costos reducidos de las no básicas

4.6 Análisis de sensibilidad

Calcula desde el tablero óptimo: **precios sombra** (coef. de holguras en fila z), **costos reducidos**, **rangos de RHS** (vía B^{-1} y razón sobre x_B) y **rangos de coeficientes objetivo** (separando básicas y no básicas).

5. Resultados sobre el problema modelo

5.1 Tablero óptimo del Simplex

Tablero óptimo — $Z^* = 36.0$, $x^* = (2.0, 6.0)$

	x1	x2	s1	s2	s3	RHS
z	0.0	0.0	0.0	1.5	1.0	36.0
s1	0.0	0.0	1.0	0.3333	-0.3333	2.0
x2	0.0	1.0	0.0	0.5	0.0	6.0
x1	1.0	0.0	0.0	-0.3333	0.3333	2.0

El programa alcanza el óptimo en **2 iteraciones**. Variables básicas: $x_1 = 2$, $x_2 = 6$, $s_1 = 2$; no básicas: $s_2 = 0$, $s_3 = 0$.

5.2 Simplex Revisado (forma matricial)

B (base óptima)

B^{-1}

$y = c_B \cdot B^{-1}$ (precios sombra)

1.000	0.000	1.000
0.000	2.000	0.000
0.000	2.000	3.000

1.000	0.333	-0.333
0.000	0.500	0.000
0.000	-0.333	0.333

0.000	1.500	1.000
-------	-------	-------

El vector dual $y = (0, 1.5, 1)$ se obtiene como $c_B \cdot B^{-1}$ y coincide con los precios sombra del análisis de sensibilidad — lo que confirma la consistencia entre ambos métodos.

5.3 Análisis de sensibilidad

Precios sombra y rangos de RHS (b)

	Precio sombra	b mín	b máx
Planta 1	0.0	2.0	inf
Planta 2	1.5	6.0	18.0
Planta 3	1.0	12.0	24.0

Lectura:

- **Planta 1** tiene precio sombra 0 $\rightarrow$ la restricción $x_1 \leq 4$ no está activa (sobra capacidad: $s_1 = 2$). Un aumento de su disponibilidad no mejora Z.
- **Planta 2** tiene precio sombra 1.5 $\rightarrow$ cada unidad adicional de tiempo en esa planta aumentaría Z^* en 1.5, dentro del rango $b_2 \in [6, 18]$.
- **Planta 3** tiene precio sombra 1.0 $\rightarrow$ cada unidad adicional aumenta Z^* en 1.0, dentro del rango $b_3 \in [12, 24]$.

Costos reducidos y rangos de c

	Costo reducido	c mín	c máx
x1	0.0	0.0	7.5
x2	0.0	2.0	inf

Ambas variables x_1 y x_2 son básicas (costo reducido = 0). Los rangos indican cuánto puede variar cada coeficiente sin que cambie la base óptima.

6. Casos especiales detectados

Caso	Cómo se detecta
Óptimo único	Todos los costos reducidos > 0 al converger
Óptimos múltiples	Existe variable no básica con costo reducido = 0
No acotado	La columna entrante no tiene coeficientes positivos para la razón mínima
Infactible	(Gran M) Alguna artificial básica queda con valor > 0
Degeneración	Empate en la razón mínima $\rightarrow$ se aplica regla de Bland

7. Validación

Suite de tests en `tests/test_all.py`, todos pasando:

Problema	Resultado esperado	Estado
----------	--------------------	--------

Wyndor Glass (Hillier)	$Z=36, x=(2, 6)$	✓
Dieta (mixto $\leq/\geq$)	$Z=5.25, x=(7.5, 4.5)$	✓
$\max x_1+x_2$ con $-x_1+x_2\leq 1$	No acotado	✓
$x_1+x_2\leq 2 \wedge x_1+x_2\geq 5$	Infactible	✓
Precios sombra Wyndor	$(0, 1.5, 1)$	✓
Rango RHS Planta 2	$[6, 18]$	✓
Simplex Revisado (Wyndor)	$y = (0, 1.5, 1)$	✓

8. Cómo ejecutar el programa

```
# Una sola vez
python3 -m venv .venv
.venv/bin/pip install -r requirements.txt

# Lanzar la UI
.venv/bin/streamlit run app.py      # http://localhost:8501

# Correr los tests
.venv/bin/python tests/test_all.py
```

Tecnologías: Python 3.13 · NumPy · pandas · Streamlit · matplotlib.